

Sistemas de Información

Práctica 3

Cardiel, Enrique
874417@unizar.es

Sierra, Víctor
869244@unizar.es

Blanchard, Daniel
870373@unizar.es

September 2025

Contents

1	Introducción	3
2	Diseño y Creación de la Base de Datos	3
3	Desarrollo de la Capa de Datos	6
4	Decisiones Tomadas	8
5	Metodología	9
6	Dificultades Encontradas	10
7	Bibliografía	10

1 Introducción

En esta práctica nos vamos a centrar en la creación de la base de datos para Yami. Esto engloba desde un primer boceto hasta la implementación del VO que nos permita interactuar con nuestra base de datos.

Además, diseñaremos un esquema Entidad-Relación, definiremos las restricciones de nuestras tablas y profundizaremos en cómo están relacionadas unas con otras mediante el uso de foreign keys.

En adición, también reflejaremos distintas decisiones que hemos tomado respecto a Yami en cuanto a sus planes de negocio, sus características y demás.

2 Diseño y Creación de la Base de Datos

Para crear la base de datos hemos optado por la página **Supabase**, una de las que se encuentra en el guión de la práctica, mientras que para el código de VO y DAO se ha utilizado **Python**. Primero comenzamos haciendo una reflexión de qué tablas íbamos a necesitar, cómo se iban a relacionar entre ellas y qué campos iban a tener.

Finalmente, decidimos que nuestra base de datos va a contar con las siguientes tablas:

Tabla Users

Tabla en la que se guardará a clientes, restaurantes y administradores. Contiene sus credenciales para iniciar sesión:

- **id**: Es la primary key, recoge el id asignado a cada usuario. Es independiente de su rol, tipo int.
- **email***: Correo electrónico del usuario, tipo varchar.
- **password**: Contraseña del usuario. Esta columna será cifrada para cumplir con la protección de datos, tipo varchar. *Restricción*: longitud mínima de 8 caracteres.
- **role**: Rol en Yami (Cliente, Restaurante o Administrador), tipo de dato enumerado *Roles* que toma los valores Admin, Client, Restaurant.

Tabla Clients

Recoge todos los datos que introducen los Clientes al registrarse en Yami. Las columnas de la tabla son las siguientes:

- **id**: Es la primary key, recoge el id asignado a cada cliente, tipo int.
- **user_id**: Es la foreign key, está relacionada con la primary key de la tabla Users.
- **name**: Nombre del cliente, tipo varchar.
- **surname**: Apellidos del cliente, tipo varchar.

- **address:** Dirección en la que realizar las entregas, tipo varchar.
- **city:** Ciudad, tipo varchar.
- **postal_code:** Código postal, tipo varchar.
- **dni:** DNI del cliente, tipo varchar.
- **phone_number:** Número de teléfono que se usará para contactar en caso de incidencia, tipo varchar. *Restricción:* longitud menor de diez caracteres.
- **available_credits:** Créditos disponibles, tipo int.
- **subscription_renewal_date:** Fecha de suscripción a Yami, tipo timestamp.
- **sub_plan:** Foreign key que relaciona la columna con la columna name de la tabla Subscription Plans.

Tabla Restaurants

Recoge todos los datos que introducen los Restaurantes al registrarse en Yami. Las columnas de la tabla son las siguientes:

- **id:** Es la primary key, recoge el id asignado a cada restaurante, tipo int.
- **user_id:** Foreign key relacionada con la primary key de la tabla Users.
- **name*:** Nombre del restaurante, tipo varchar.
- **description:** Descripción del restaurante, cuál es su oferta, etc. Tipo varchar.
- **city:** Ciudad, tipo varchar.
- **address*:** Dirección en la que se encuentra el local, tipo varchar.
- **phone_number:** Número de teléfono que se usará para contactar en caso de incidencia, tipo varchar. *Restricción:* longitud menor de diez caracteres.
- **category:** Categoría en la que encaja (italiano, japonés, etc.), tipo de dato enumerado *Restaurant_Categories*. Actualmente se puede elegir de entre las siguientes opciones: Italiano, Asiático, Hamburguesería, Turco, Asador, Comida Rápida, Mexicano, Indio, Griego, Bocatería, Saludable, Vegana, Heladería.

Tabla Dish Types

Recoge los diferentes tipos de platos que se pueden ofrecer en Yami:

- **id:** Es la primary key, recoge el id asignado a cada tipo de plato que se ofrece, tipo int.
- **name*:** Nombre del tipo de plato. Esta columna es de un tipo de dato enumerado *dish_types*. Actualmente puede tomar los siguientes valores: Entrante, Principal, Postre, Bebida.
- **credits:** Numero de créditos que cuesta el tipo de plato. Tipo int.

Tabla Dishes

Recoge todos los datos de los platos que ofrece cada restaurante, contando con las siguientes columnas:

- **id**: Es la primary key, recoge el id asignado a cada restaurante, tipo int.
- **restaurant_id**: Es la foreign key, está relacionada con la primary key de la tabla Restaurants. Hace referencia al restaurante que ofrece este plato.
- **name**: Nombre del plato, tipo varchar.
- **description**: Descripción del plato, su elaboración, ingredientes, etc. Tipo varchar.
- **allergens**: Alérgenos que contiene el plato, tipo varchar.
- **dish_type**: Categoría del plato en Yami (entrante, plato principal, postre o bebida), tipo de dato enumerado *dish_types*. Foreign Key a la tabla Dish Types.

Tabla Subscription Plans

Recoge los planes de suscripción que ofrece Yami:

- **id**: Es la primary key, recoge el id asignado a cada plan, tipo int.
- **credits**: Número total de créditos que ofrece el plan, tipo int.
- **name***: Nombre del plan, tipo de dato enumerado *subscription_names* que toma los siguientes valores: Basic, Plus, Deluxe.
- **precio**: Precio del plan, tipo float.

Tabla Orders

Recoge los pedidos realizados en nuestra página web:

- **id**: Es la primary key, recoge el id asignado a cada pedido, tipo int.
- **client_id**: Foreign key que relaciona la columna con el id del cliente que realiza el pedido.
- **restaurant_id**: Foreign key que relaciona la columna con el id del restaurante que recibe el pedido.
- **order_credits**: Número total de créditos gastados en el pedido, tipo int.
- **order_date**: Fecha en la que se realizó el pedido, tipo timestamp.
- **order_status**: Estado en el que se encuentra un pedido. Tipo de dato enumerado *order_status* que toma los siguientes valores: Encargado, En preparacion, En reparto, Entregado, Cancelado.

Tabla Ordered Dishes

Recoge los platos que contiene cada pedido:

- **id**: Es la primary key, recoge el id asignado a cada plan, tipo int.
- **order_id**: Foreign key que relaciona la tabla con la tabla Orders, hace referencia al campo id de la misma.
- **dish_id**: Id del plato pedido, es una foreign key que relaciona la tabla con la tabla Dishes, hace referencia al campo id de la misma.
- **dish_name**: Nombre del plato pedido, tipo varchar.
- **instructions**: Instrucciones especificadas, tipo varchar.

Tabla Ratings

Recoge las puntuaciones que los clientes ponen a los restaurantes:

- **id**: Es la primary key, recoge el id asignado a cada puntuación, tipo int.
- **client_id**: Foreign key que relaciona la columna con el id del cliente que realiza el pedido.
- **restaurant_id**: Foreign key que relaciona la columna con el id del restaurante que recibe el pedido.
- **rating**: Puntuación de 1 a 5, tipo int.

Los elementos marcados con * cuentan con una restricción *unique*.

3 Desarrollo de la Capa de Datos

Antes de comenzar con el diseño de la base de datos hemos creado un modelo Entidad-Relación (E-R) que nos permite visualizar las tablas que vamos a emplear y como están relacionadas entre si. Para diseñar el modelo E-R hemos utilizado DBMain, software que empleamos el curso pasado en la asignatura de *Bases de Datos* para elaborar esquemas E-R.

El modelo E-R nos permite visualizar la cardinalidad entre las diferentes tablas, por ejemplo, en la relacion Users/Clients podemos observar como cada user puede estar relacionado a cero o a un cliente mientras que cada cliente tiene un unico user. Los identificadores de cada tabla son los campos *id* que aparecen subrayados y debajo de todos los atributos. De esta forma hemos conseguido el diagrama que se puede ver en la figura 1.

Una vez construido el modelo Entidad-Relación, gracias a *DBmain*, podemos convertirlo en un modelo Relacional. El modelo relacional nos da aun mas información acerca de como están relacionadas las tablas, pudiendo ver tanto las claves primarias como las claves foráneas entre tablas. Este diagrama 2 es el que hemos empleado para crear las tablas en **Supabase**.

Para el diseño de nuestra base de datos es muy importante tener las tablas relacionadas mediante claves foráneas debido a que todas las tablas mantienen relaciones entre si y los datos tiene que ser consistentes. Por ejemplo, en el caso de que un usuario haga un pedido con un numero N de pedidos necesitamos que el pedido este relacionado tanto con el cliente, como con el restaurante del que quiere pedir la comida. A su vez cada plato que ha pedido esta relacionado con el pedido total y con el restaurante que ofrece el plato. Tanto el cliente como el restaurante deben tener un Usuario luego, y en el caso del cliente tiene que tener asociado un tipo de plan al que esta suscrito. Por lo tanto, indirectamente todas las tablas de nuestra base de datos están relacionadas entre si y es de extrema importancia tener un buen modelo relacional que nos permita fácilmente visualizar las relaciones.

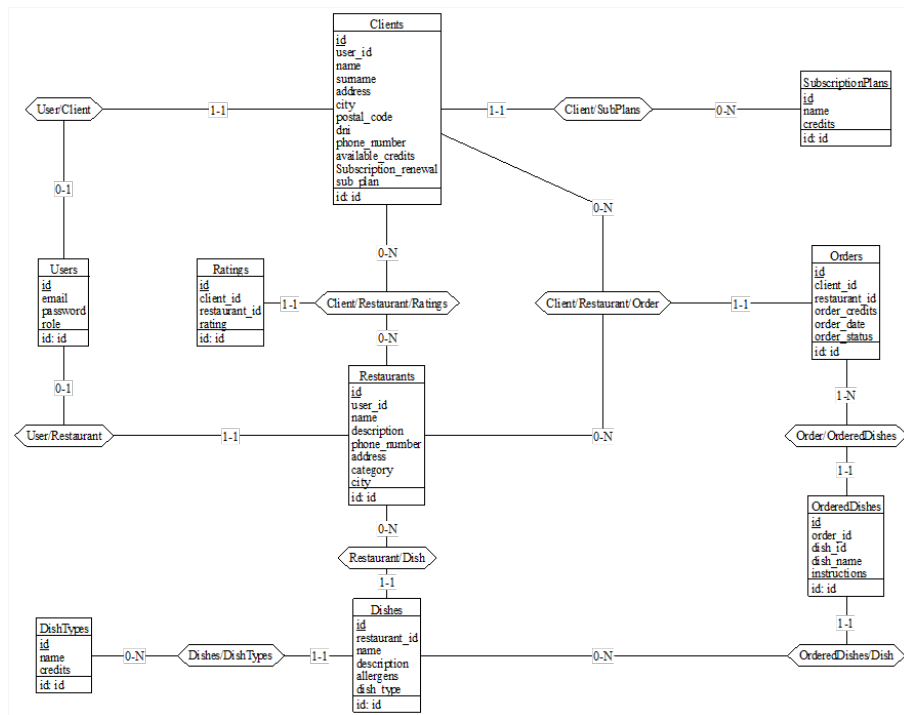


Figure 1: Modelo Entidad-Relación

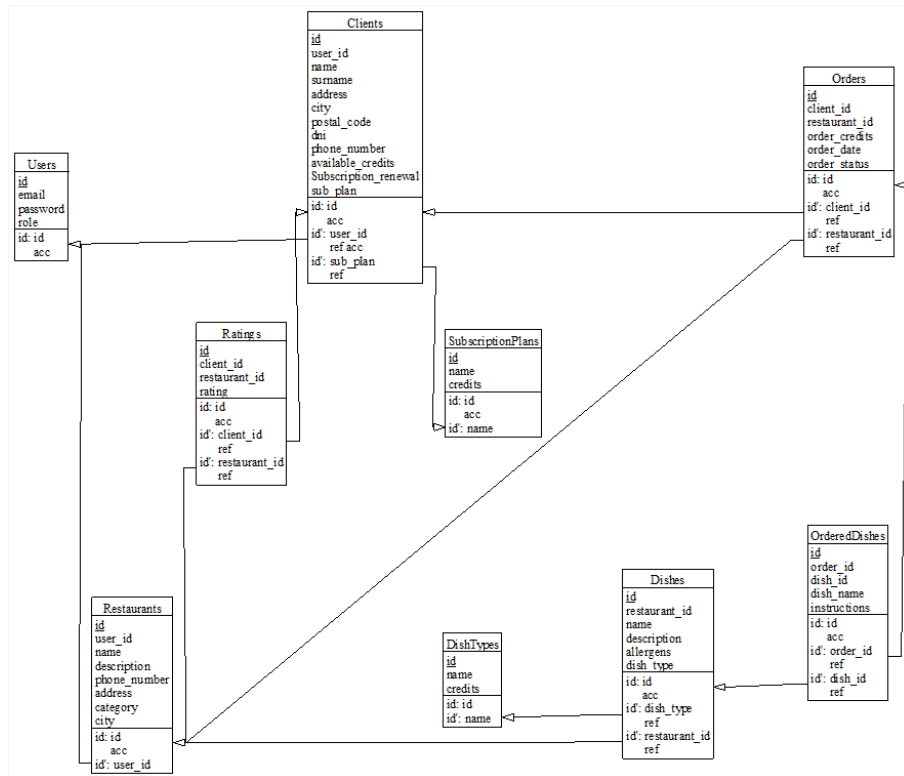


Figure 2: Modelo Relacional

4 Decisiones Tomadas

Tras una reunión grupal, hemos decidido que los créditos que ofrece cada plan van a recibir el nombre de **Yameats**. Además, hemos establecido los siguientes precios para los artículos de la carta que ofrece Yami:

- **Bebida:** 1 Yameat.
- **Entrante o Primer Plato:** 3 Yameats.
- **Plato Principal:** 5 Yameats.
- **Postre:** 3 Yameats.

Además, también hemos tomado la decisión de cambiar el número de créditos que ofrece cada plan con el objetivo de que vaya en concordancia con los precios. El resultado es el siguiente:

Plan	Yameats	Precio
Básico	30	63,99€
Plus	60	129,99€
Deluxe	100	209,99€

Como se puede observar, también le hemos asignado un precio a cada plan. Para ello hemos echado cuentas y decidido buscando la prosperidad económica.

5 Metodología

En esta tercera práctica hemos seguido con la metodología de las dos prácticas anteriores. En esta practica hemos rotado el "líder", en este caso el encargado de liderar la practica ha sido **Víctor Sierra** y ha sido quien ha realizado la entrega.

En concreto, la división de tareas ha sido la siguiente:

- Víctor Sierra (6,75 horas):
 1. Introducción de la Memoria (0.25 horas)
 2. Descripción de las Tablas en la Memoria (1,5 horas)
 3. Creación de Tablas en Supabase (1,5 horas)
 4. Completar Diseño VO y DAO (1 hora)
 5. Testeo de Database (2,5 horas)
- Daniel Blanchard (7 horas):
 4. Memoria (0,5 horas)
 5. Modelo Entidad-Relación y modelo relacional (2 hora)
 6. Diseño VO y DAO (3 horas)
 7. Elaboración de funciones generales (Creación de Cliente, Restaurante, etc ...) (1,5 horas)
 8. Testeo de Database (3 horas)
- Enrique Cardiel (4 horas):
 9. Revisión de la memoria (0,5 horas)
 10. Creación de tablas y restricciones en Supabase (1,5 horas)
 11. Apartado Diseño y Creación de la Base de Datos (1 horas)
 12. Corrección de elementos e incongruencias de la práctica 2 (1 hora)

Para realizar los tests la base de datos, se ha utilizado el script *test.py*. Concretamente, este fichero prueba:

- Creación de clientes y restaurantes (incluye usuario).
- Creación de platos y pedidos.
- Creación de parejas (usuario restaurante) de ratings .
- Actualización de información de cliente, créditos de cliente, información de restaurante e información de platos.

Además, para facilitar los test se ha creado una función (`clean_db()`) que borra toda la información del sistema, excepto los usuarios tipo *admin* y los tipos de planes. Como test negativo, hemos realizado una ejecución del script `test.py` para crear las tablas. Después, comentando la función `clean_db()` y ejecutándolo de nuevo, aparecen los errores esperados, como por ejemplo, al intentar crear un cliente con mismo email aparece un error debido a la restricción de unicidad:

```
Error creating client: {'message': 'duplicate key value violates unique constraint "users_password_email"', 'code': '23505', 'hint': None, 'details': 'Key (email)=(alice@example.com) already exists.'}
```

6 Dificultades Encontradas

Por lo general el desarrollo de esta práctica nos ha resultado ligeramente más sencillo de lo esperado debido a la acertada elección de usar **Supabase**, que nos permitió plasmar con facilidad las ideas que teníamos. La única dificultad que se podría destacar, fue a la hora de conectar el código a **Supabase** a través de su API, debido a que tuvimos que configurar los permisos adecuados para que nuestro *schema* fuera visible desde el código.

7 Bibliografía

References

- [1] Supabase *Uso de enum types en Supabase*, Disponible en: <https://supabase.com/docs/guides/database/postgres/enums>, último acceso: October 16, 2025.